

Step-by-Step Guide

VPC Peering on AWS Using Ubuntu EC2 Instances

A complete hands-on walkthrough — from creating two VPCs to verifying cross-VPC connectivity between Ubuntu servers

Prepared by: **ADITYA**

Environment: AWS Management Console | AMI: Ubuntu Server 22.04
LTS

Region used in examples: us-east-1 (N. Virginia)

All resource names in this guide follow the convention **<ResourceType>_ADITYA** so that they are easy to identify in a shared or training AWS account.

Table of Contents

1. Introduction to VPC Peering
2. Architecture Overview & Naming Convention
3. Prerequisites
4. Step 1 — Create Two VPCs (VPC1_ADITYA & VPC2_ADITYA)
5. Step 2 — Create Subnets in Each VPC
6. Step 3 — Create and Attach an Internet Gateway
7. Step 4 — Launch Ubuntu EC2 Instances
8. Step 5 — Configure Security Groups
9. Step 6 — Create the VPC Peering Connection
10. Step 7 — Accept the Peering Connection
11. Step 8 — Update Route Tables
12. Step 9 — Test Connectivity Between Instances
13. Troubleshooting Common Issues
14. Cleanup — Deleting Resources
15. Summary & Best Practices

1. Introduction to VPC Peering

A **VPC Peering Connection** is a networking connection between two Amazon Virtual Private Clouds (VPCs) that enables instances in either VPC to communicate with each other using private IPv4 or IPv6 addresses, as if they were on the same network. AWS uses the existing infrastructure of a VPC to create the peering connection — traffic never traverses the public internet, and no gateway, VPN, or separate physical hardware is required.

VPC Peering is commonly used to:

- Connect VPCs that belong to different applications, teams, or environments (e.g. Dev and Prod).
- Share resources (databases, internal APIs) between VPCs without exposing them publicly.
- Connect VPCs across different AWS accounts or different regions (inter-region peering).
- Build multi-tier architectures where each tier is isolated in its own VPC for security.

Note: VPC Peering connections are **not transitive**. If VPC A is peered with VPC B, and VPC B is peered with VPC C, VPC A cannot automatically reach VPC C. A separate peering connection (or Transit Gateway) is required for that.

In this guide, we will create two VPCs — **VPC1_ADITYA** and **VPC2_ADITYA** — each with a single public subnet and one Ubuntu Server 22.04 LTS EC2 instance. We will then peer the two VPCs together, update the route tables and security groups, and verify connectivity by pinging one instance from the other over their private IP addresses.

2. Architecture Overview & Naming Convention

The diagram below shows the target architecture we will build in this guide. Every resource name is suffixed with **_ADITYA** so it can be quickly identified.

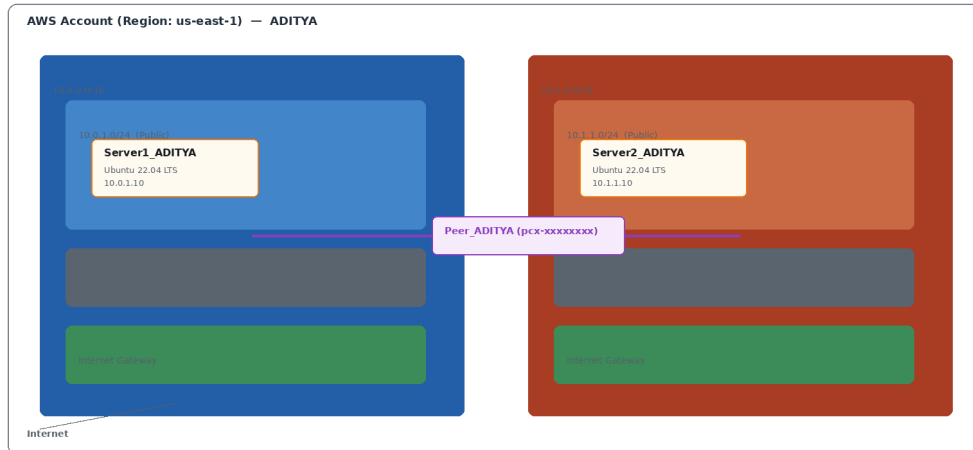


Figure 2.1 — Target architecture: two peered VPCs, each with a public subnet, an Ubuntu instance, an Internet Gateway, and a route table.

Naming convention used throughout this guide

Resource Type	Name Used
VPC (Network 1)	VPC1_ADITYA
VPC (Network 2)	VPC2_ADITYA
Subnet (Network 1)	Subnet1_ADITYA
Subnet (Network 2)	Subnet2_ADITYA
Internet Gateway 1	IGW1_ADITYA
Internet Gateway 2	IGW2_ADITYA
EC2 Instance 1 (Ubuntu)	Server1_ADITYA
EC2 Instance 2 (Ubuntu)	Server2_ADITYA
Security Group 1	SG1_ADITYA
Security Group 2	SG2_ADITYA
Route Table 1	RouteTable1_ADITYA
Route Table 2	RouteTable2_ADITYA

Peering Connection	Peer_ADITYA
--------------------	-------------

3. Prerequisites

- An active AWS account with permissions to create VPC, EC2, and IAM-related resources.
- AWS Management Console access (console.aws.amazon.com).
- A key pair for SSH access to EC2 instances (we will use **AdityaKey.pem**).
- Basic familiarity with CIDR notation and Linux terminal commands.
- An SSH client — Terminal (macOS/Linux) or PuTTY / Windows Terminal (Windows).

Note: This guide uses non-overlapping CIDR blocks — **10.0.0.0/16** for VPC1_ADITYA and **10.1.0.0/16** for VPC2_ADITYA. VPC Peering requires that the two VPC CIDR blocks do not overlap.

4. Step 1 — Create Two VPCs

Navigate to the **VPC Dashboard** in the AWS Management Console and select **Your VPCs** → **Create VPC**. We will repeat this step twice to create both VPCs.

4.1 Create VPC1_ADITYA

- Choose **VPC only** as the creation option.
- Name tag: **VPC1_ADITYA**
- IPv4 CIDR block: **10.0.0.0/16**
- Tenancy: Default
- Click **Create VPC**.

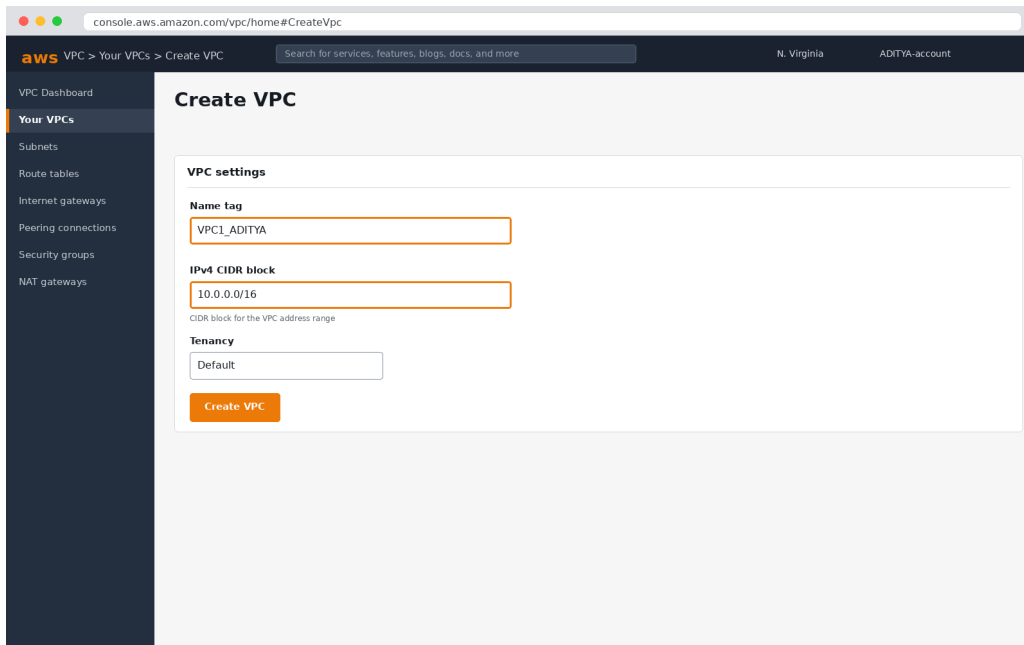


Figure 4.1 — Creating VPC1_ADITYA with CIDR block 10.0.0.0/16.

4.2 Create VPC2_ADITYA

- Repeat the same process.
- Name tag: **VPC2_ADITYA**
- IPv4 CIDR block: **10.1.0.0/16**
- Click **Create VPC**.

The screenshot shows the 'Create VPC' page in the AWS Management Console. The left sidebar contains navigation links: VPC Dashboard, Your VPCs (selected), Subnets, Route tables, Internet gateways, Peering connections, Security groups, and NAT gateways. The main content area is titled 'Create VPC' and contains a 'VPC settings' section. This section has three input fields: 'Name tag' with the value 'VPC2_ADITYA', 'IPv4 CIDR block' with the value '10.1.0.0/16', and 'Tenancy' with the value 'Default'. Below these fields is an orange 'Create VPC' button.

Figure 4.2 — Creating VPC2_ADITYA with CIDR block 10.1.0.0/16.

Once both VPCs are created, the **Your VPCs** list should show both entries in the **Available** state, as shown below.

The screenshot shows the 'Your VPCs' page in the AWS Management Console. It features a table with the following data:

Name	VPC ID	State	IPv4 CIDR
VPC1_ADITYA	vpc-0a1b2c3d4e5f6g7h1	Available	10.0.0.0/16
VPC2_ADITYA	vpc-0a1b2c3d4e5f6g7h2	Available	10.1.0.0/16

Figure 4.3 — Both VPCs listed as Available in the VPC dashboard.

5. Step 2 — Create Subnets in Each VPC

Each VPC needs at least one subnet before an EC2 instance can be launched inside it. Go to **VPC** → **Subnets** → **Create subnet**.

5.1 Create Subnet1_ADITYA (inside VPC1_ADITYA)

- VPC ID: **VPC1_ADITYA**
- Subnet name: **Subnet1_ADITYA**
- Availability Zone: **us-east-1a**
- IPv4 CIDR block: **10.0.1.0/24**
- Click **Create subnet**.

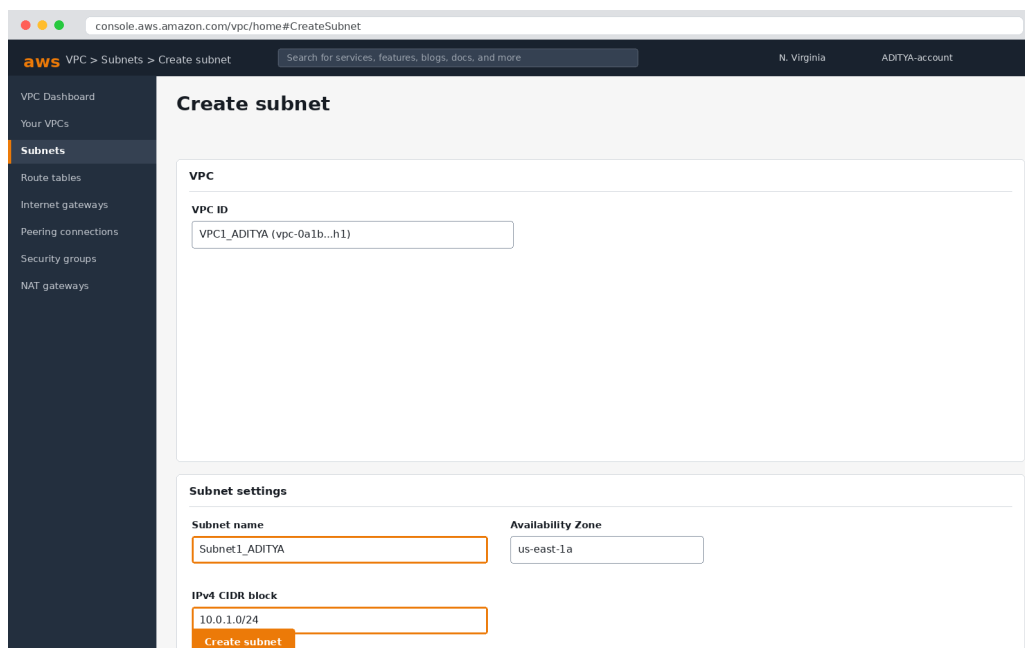
The screenshot shows the AWS Management Console interface for creating a new subnet. The browser address bar displays 'console.aws.amazon.com/vpc/home#CreateSubnet'. The left-hand navigation pane is open, showing the 'Subnets' section under the 'VPC' category. The main content area is titled 'Create subnet'. It features a 'VPC ID' field with the value 'VPC1_ADITYA (vpc-0a1b...h1)' selected. Below this, the 'Subnet settings' section contains three input fields: 'Subnet name' with 'Subnet1_ADITYA', 'Availability Zone' with 'us-east-1a', and 'IPv4 CIDR block' with '10.0.1.0/24'. A blue 'Create subnet' button is positioned at the bottom of the form.

Figure 5.1 — Creating Subnet1_ADITYA (10.0.1.0/24) inside VPC1_ADITYA.

5.2 Create Subnet2_ADITYA (inside VPC2_ADITYA)

- VPC ID: **VPC2_ADITYA**
- Subnet name: **Subnet2_ADITYA**
- Availability Zone: **us-east-1a**
- IPv4 CIDR block: **10.1.1.0/24**
- Click **Create subnet**.

The screenshot shows the AWS Management Console interface for creating a new subnet. The breadcrumb navigation at the top indicates the path: VPC > Subnets > Create subnet. The left-hand navigation menu includes links to VPC Dashboard, Your VPCs, Subnets (which is highlighted), Route tables, Internet gateways, Peering connections, Security groups, and NAT gateways. The main content area is titled 'Create subnet'. It features a 'VPC ID' field containing 'VPC2_ADITYA (vpc-0a1b...h2)'. Below this, the 'Subnet settings' section contains three input fields: 'Subnet name' with the value 'Subnet2_ADITYA', 'Availability Zone' with the value 'us-east-1a', and 'IPv4 CIDR block' with the value '10.1.1.0/24'. An orange 'Create subnet' button is positioned at the bottom of the settings section.

Figure 5.2 — Creating Subnet2_ADITYA (10.1.1.0/24) inside VPC2_ADITYA.

Note: After creating each subnet, open the subnet's **Actions** → **Edit subnet settings** and enable **Auto-assign public IPv4 address** so the Ubuntu instances receive a public IP automatically for SSH access.

6. Step 3 — Create and Attach an Internet Gateway

To allow SSH access to our Ubuntu instances from the internet, each VPC needs an Internet Gateway (IGW) attached to it. Go to **VPC** → **Internet gateways** → **Create internet gateway**.

6.1 Create IGW1_ADITYA

- Name tag: **IGW1_ADITYA**
- Click **Create internet gateway**.

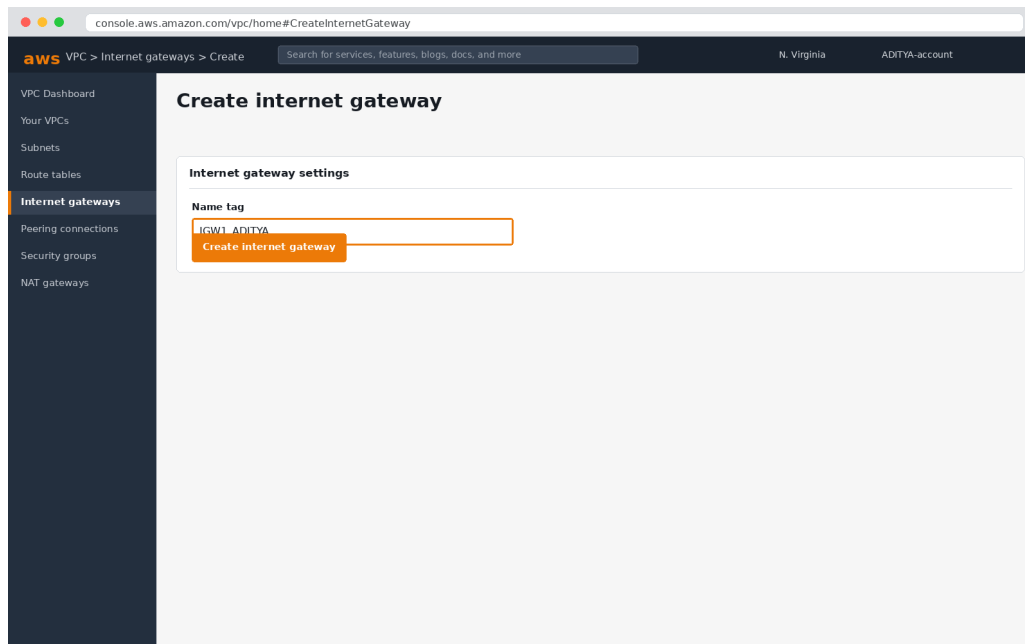


Figure 6.1 — Creating IGW1_ADITYA.

6.2 Attach IGW1_ADITYA to VPC1_ADITYA

- Select IGW1_ADITYA, then choose **Actions** → **Attach to VPC**.
- Available VPC: **VPC1_ADITYA**
- Click **Attach internet gateway**.

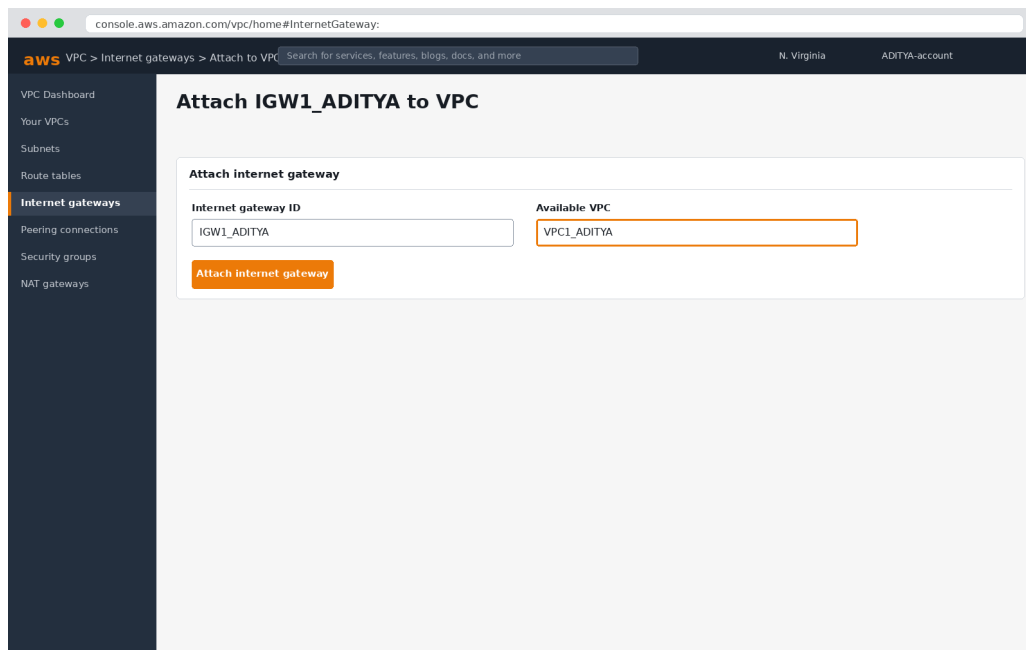


Figure 6.2 — Attaching IGW1_ADITYA to VPC1_ADITYA.

Repeat the exact same steps to create **IGW2_ADITYA** and attach it to **VPC2_ADITYA**. Both VPCs must have their own Internet Gateway for public internet access — this is separate from and unrelated to the VPC peering connection we create later, which only handles private, inter-VPC traffic.

7. Step 4 — Launch Ubuntu EC2 Instances

Navigate to **EC2** → **Instances** → **Launch an instance**. We will launch one Ubuntu instance in each VPC.

7.1 Launch Server1_ADITYA in VPC1_ADITYA

- Name: **Server1_ADITYA**
- AMI: **Ubuntu Server 22.04 LTS** (64-bit x86, Free tier eligible)
- Instance type: **t2.micro**
- Key pair: **AdityaKey.pem** (create a new one if you don't have it)
- Network (VPC): **VPC1_ADITYA**
- Subnet: **Subnet1_ADITYA**
- Auto-assign public IP: **Enable**
- Click **Launch instance**.

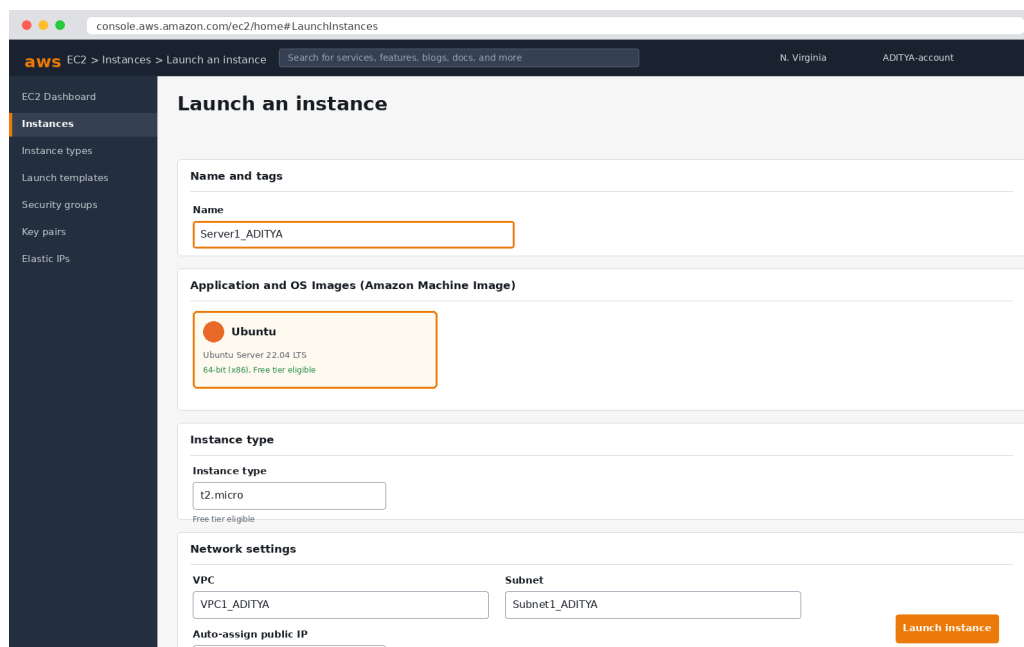


Figure 7.1 — Launching Server1_ADITYA (Ubuntu 22.04 LTS) into VPC1_ADITYA / Subnet1_ADITYA.

7.2 Launch Server2_ADITYA in VPC2_ADITYA

- Name: **Server2_ADITYA**
- AMI: **Ubuntu Server 22.04 LTS**
- Instance type: **t2.micro**
- Key pair: **AdityaKey.pem**
- Network (VPC): **VPC2_ADITYA**
- Subnet: **Subnet2_ADITYA**
- Auto-assign public IP: **Enable**
- Click **Launch instance**.

console.aws.amazon.com/ec2/home#LaunchInstances

aws EC2 > Instances > Launch an instance

Search for services, features, blogs, docs, and more

N. Virginia ADITYA-account

EC2 Dashboard

Instances

Instance types

Launch templates

Security groups

Key pairs

Elastic IPs

Launch an instance

Name and tags

Name

Server2_ADITYA

Application and OS Images (Amazon Machine Image)

Ubuntu

Ubuntu Server 22.04 LTS

64-bit (x86), Free tier eligible

Instance type

Instance type

t2.micro

Free tier eligible

Network settings

VPC

VPC2_ADITYA

Subnet

Subnet2_ADITYA

Auto-assign public IP

Launch Instance

Figure 7.2 — Launching Server2_ADITYA (Ubuntu 22.04 LTS) into VPC2_ADITYA / Subnet2_ADITYA.

Once both instances reach the **Running** state, note down their private and public IPv4 addresses — we will need the private IPs to test peering connectivity later.

console.aws.amazon.com/ec2/home#Instances:

aws EC2 > Instances

Search for services, features, blogs, docs, and more

N. Virginia ADITYA-account

EC2 Dashboard

Instances

Instance types

Launch templates

Security groups

Key pairs

Elastic IPs

Instances

Name	Instance ID	State	Private IPv4	Public IPv4
Server1_ADITYA	i-0abc123def4567890	Running	10.0.1.10	3.91.22.101
Server2_ADITYA	i-0fed987cba0192837	Running	10.1.1.10	3.91.22.102

Figure 7.3 — Both Ubuntu instances running, with their private IPv4 addresses visible.

8. Step 5 — Configure Security Groups

By default, security groups only allow outbound traffic and inbound traffic from within the same security group. We need to explicitly allow SSH (for management) and ICMP (for our ping test) from the peer VPC's CIDR range.

8.1 Create SG1_ADITYA (attached to Server1_ADITYA)

- Security group name: **SG1_ADITYA**
- VPC: **VPC1_ADITYA**
- Inbound rule 1 — SSH, TCP, port 22, source 0.0.0.0/0 (or your own IP for better security)
- Inbound rule 2 — Custom ICMP, All, source **10.1.0.0/16** (VPC2_ADITYA's CIDR range)

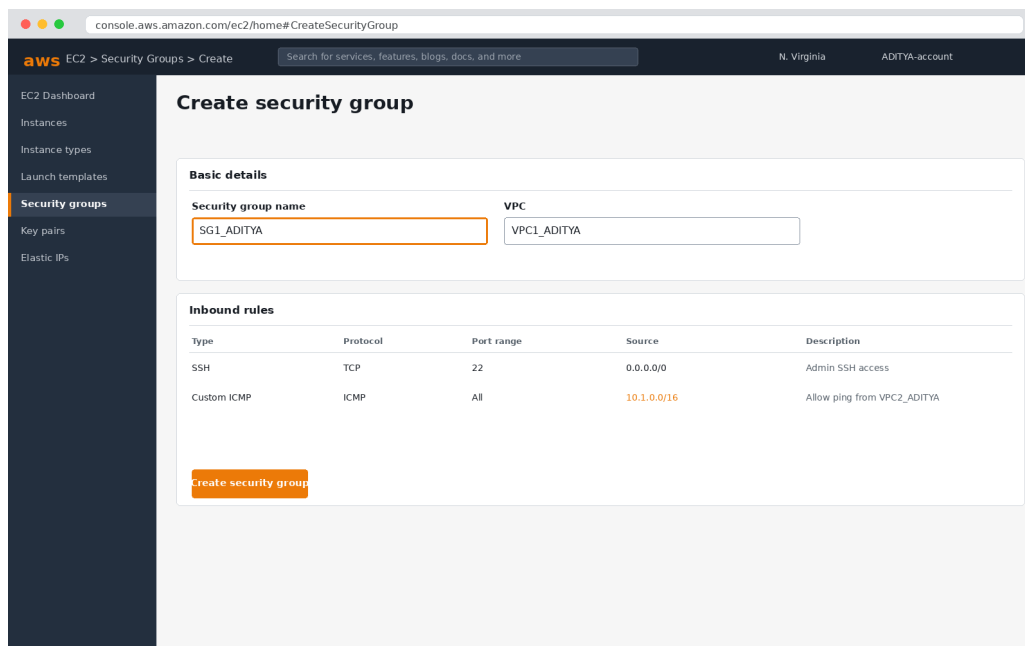


Figure 8.1 — SG1_ADITYA allowing SSH and ICMP from VPC2_ADITYA's CIDR block.

8.2 Create SG2_ADITYA (attached to Server2_ADITYA)

- Security group name: **SG2_ADITYA**
- VPC: **VPC2_ADITYA**
- Inbound rule 1 — SSH, TCP, port 22, source 0.0.0.0/0
- Inbound rule 2 — Custom ICMP, All, source **10.0.0.0/16** (VPC1_ADITYA's CIDR range)

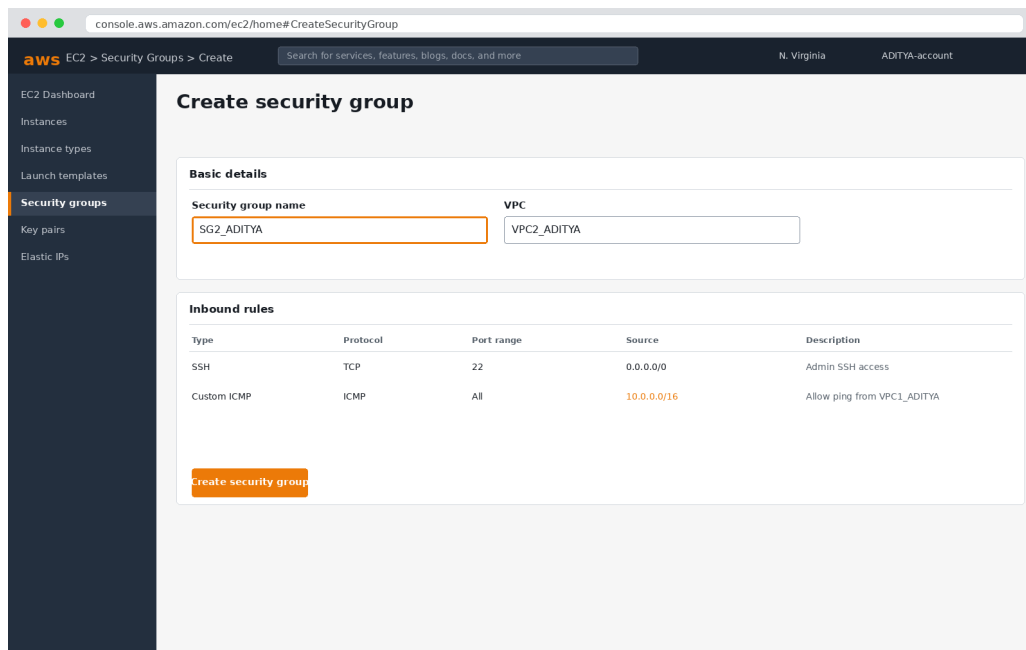


Figure 8.2 — SG2_ADITYA allowing SSH and ICMP from VPC1_ADITYA's CIDR block.

Note: Attach SG1_ADITYA to Server1_ADITYA and SG2_ADITYA to Server2_ADITYA via each instance's **Security** → **Change security groups** option, if not already selected at launch time.

9. Step 6 — Create the VPC Peering Connection

Navigate to **VPC** → **Peering connections** → **Create peering connection**.

- Peering connection name: **Peer_ADITYA**
- VPC (Requester): **VPC1_ADITYA**
- Select another VPC to peer with: **My account** → **VPC2_ADITYA** (Acceptor)
- Click **Create peering connection**.

The screenshot shows the AWS Management Console interface for creating a VPC peering connection. The breadcrumb navigation at the top reads 'VPC > Peering connections > Create'. The left-hand navigation menu includes 'VPC Dashboard', 'Your VPCs', 'Subnets', 'Route tables', 'Internet gateways', 'Peering connections' (which is highlighted), 'Security groups', and 'NAT gateways'. The main content area is titled 'Create peering connection'. It contains three input fields: 'Name' with the value 'Peer_ADITYA', 'VPC (Requester)' with the value 'VPC1_ADITYA (vpc-0a1b...h1)', and 'VPC (Acceptor)' with the value 'VPC2_ADITYA (vpc-0a1b...h2)'. Below the acceptor field, it says 'Account: 111122223333 (This account)'. An orange 'Create peering connection' button is located at the bottom of the form.

Figure 9.1 — Creating *Peer_ADITYA* between *VPC1_ADITYA* (requester) and *VPC2_ADITYA* (accepter).

After creation, the connection appears with a status of **Pending acceptance**, since both VPCs belong to the same account in this example (for cross-account peering, the owner of the acceptor VPC must approve it from their own console).

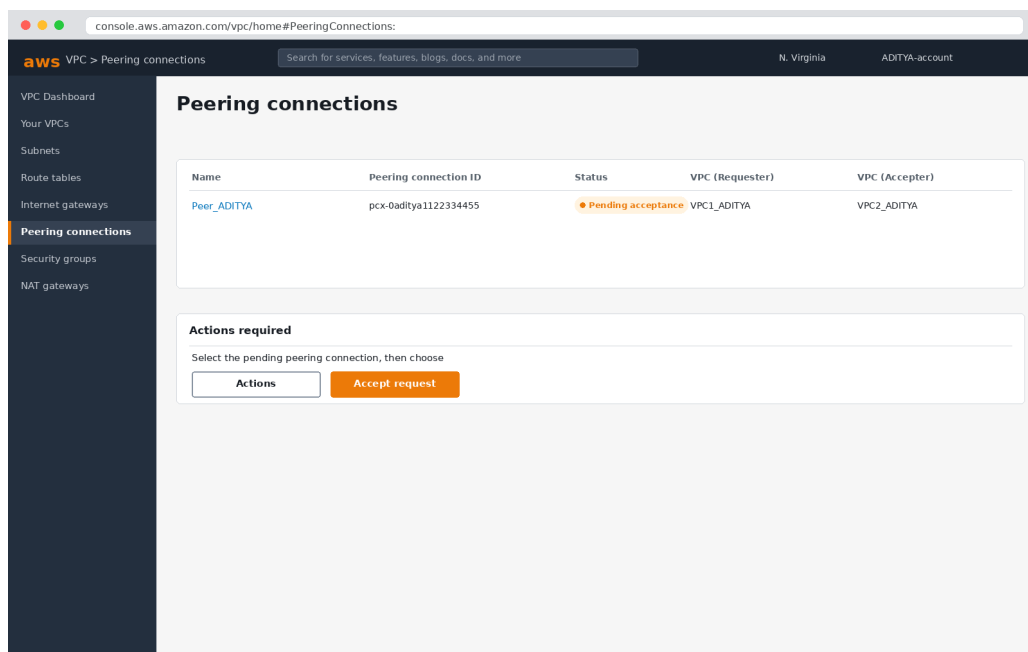


Figure 9.2 — Peer_ADITYA shown with status Pending acceptance.

10. Step 7 — Accept the Peering Connection

Select the pending peering connection **Peer_ADITYA**, then choose **Actions** → **Accept request**. Confirm the acceptance in the dialog box that appears.

- Select Peer_ADITYA in the Peering connections list.
- Choose **Actions** → **Accept request**.
- Confirm — the status changes from Pending acceptance to **Active**.

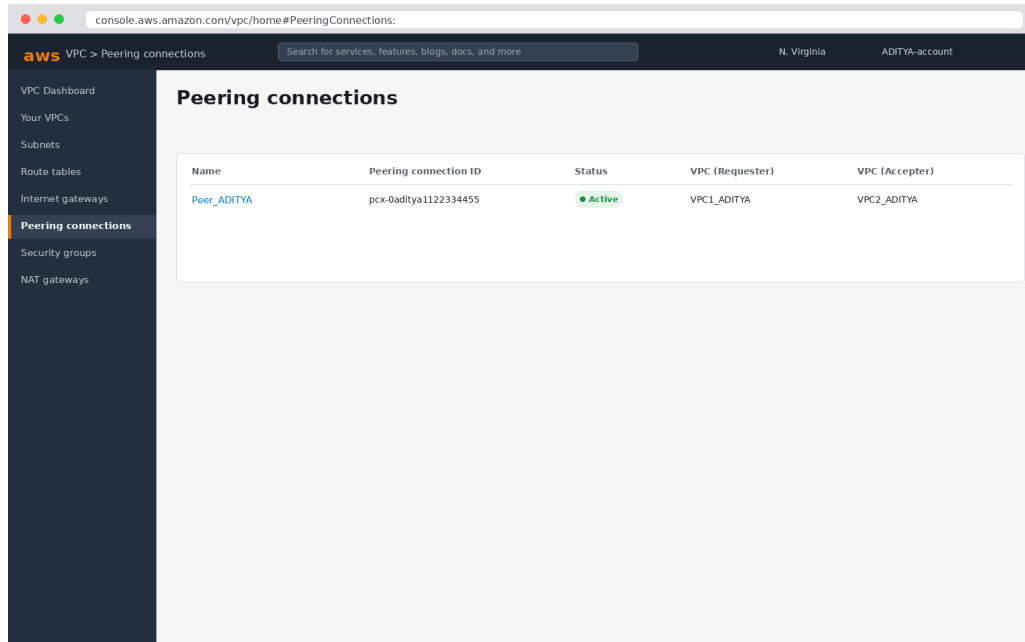


Figure 10.1 — Peer_ADITYA status changed to Active after acceptance.

Note: A VPC peering connection alone does **not** route any traffic. You must still update the route tables of both VPCs (next step) before instances can communicate.

11. Step 8 — Update Route Tables

Go to **VPC** → **Route tables**, select the route table associated with each VPC, and add a route pointing to the peering connection.

11.1 Update RouteTable1_ADITYA (VPC1_ADITYA)

- Destination: **10.1.0.0/16** (VPC2_ADITYA's CIDR)
- Target: **Peer_ADITYA** (pcx-0aditya1122334455)
- Click **Save changes**.

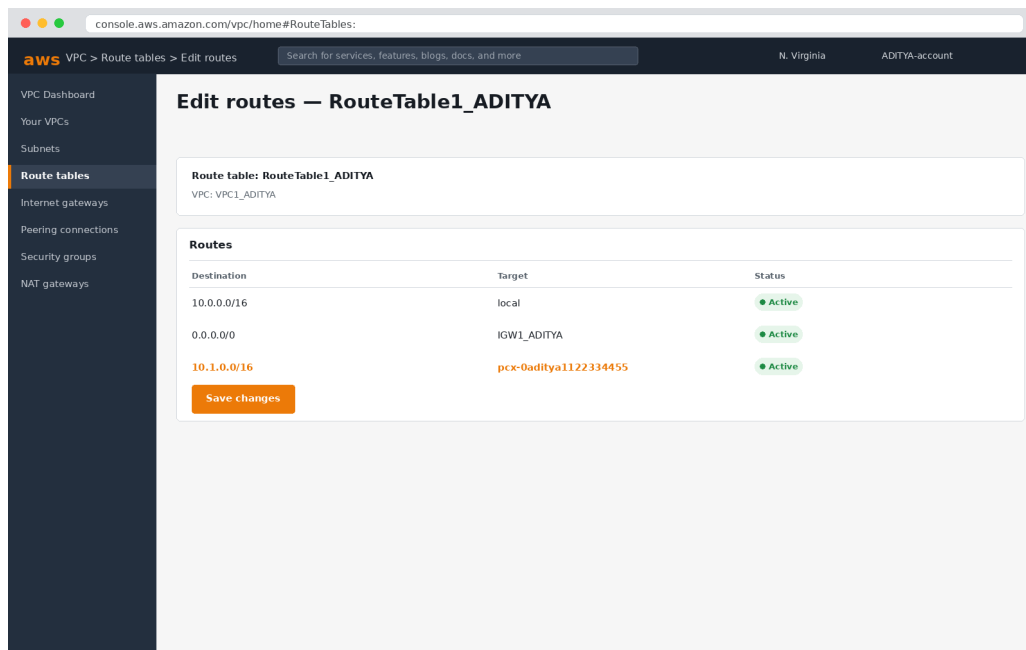


Figure 11.1 — RouteTable1_ADITYA with a new route sending 10.1.0.0/16 traffic through Peer_ADITYA.

11.2 Update RouteTable2_ADITYA (VPC2_ADITYA)

- Destination: **10.0.0.0/16** (VPC1_ADITYA's CIDR)
- Target: **Peer_ADITYA** (pcx-0aditya1122334455)
- Click **Save changes**.

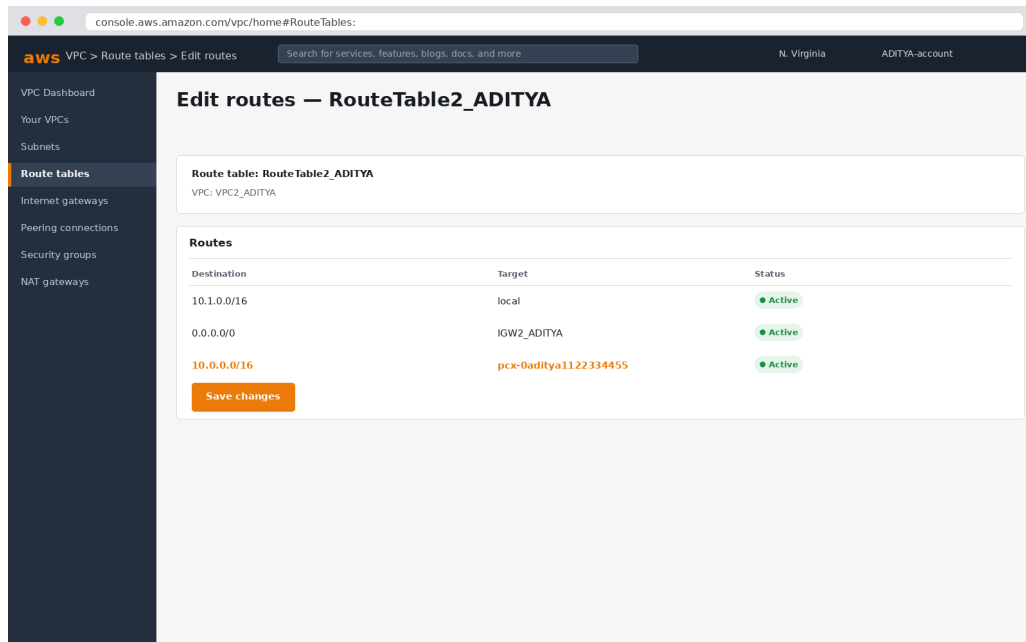
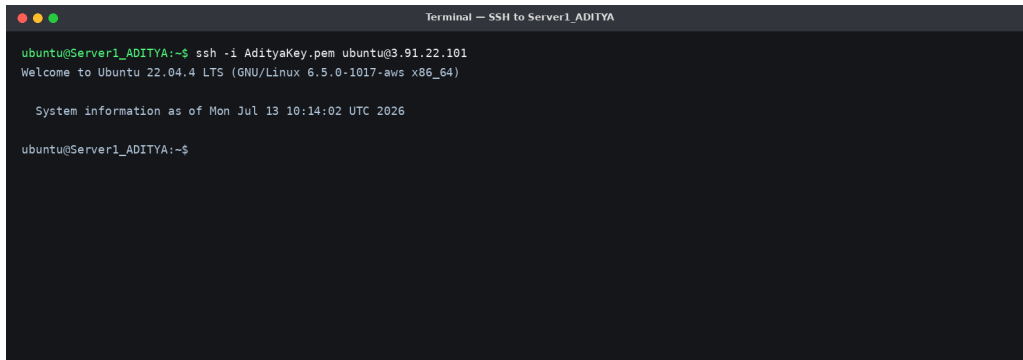


Figure 11.2 — RouteTable2_ADITYA with a new route sending 10.0.0.0/16 traffic through Peer_ADITYA.

With routes added on both sides, traffic destined for the peer VPC's CIDR block will now be routed through the peering connection instead of being dropped.

12. Step 9 — Test Connectivity Between Instances

SSH into Server1_ADITYA using its public IP address and its key pair.



```
Terminal — SSH to Server1_ADITYA

ubuntu@Server1_ADITYA:~$ ssh -i AdityaKey.pem ubuntu@3.91.22.101
Welcome to Ubuntu 22.04.4 LTS (GNU/Linux 6.5.0-1017-aws x86_64)

System information as of Mon Jul 13 10:14:02 UTC 2026

ubuntu@Server1_ADITYA:~$
```

Figure 12.1 — SSH session established with Server1_ADITYA.

From inside Server1_ADITYA, ping the **private** IPv4 address of Server2_ADITYA (10.1.1.10 in this example). If the peering connection, route tables, and security groups are configured correctly, you should see successful replies.



```
Terminal — Testing VPC Peering Connectivity

ubuntu@Server1_ADITYA:~$ ping -c 4 10.1.1.10
PING 10.1.1.10 (10.1.1.10) 56(84) bytes of data:
64 bytes from 10.1.1.10: icmp_seq=1 ttl=64 time=0.612 ms
64 bytes from 10.1.1.10: icmp_seq=2 ttl=64 time=0.487 ms
64 bytes from 10.1.1.10: icmp_seq=3 ttl=64 time=0.501 ms
64 bytes from 10.1.1.10: icmp_seq=4 ttl=64 time=0.495 ms

--- 10.1.1.10 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss

ubuntu@Server1_ADITYA:~$
```

Figure 12.2 — Successful ping from Server1_ADITYA to Server2_ADITYA over the VPC peering connection.

You can also test in the reverse direction — SSH into Server2_ADITYA and ping Server1_ADITYA's private IP (10.0.1.10) — to confirm bidirectional connectivity.

```
# From Server1_ADITYA
ssh -i AdityaKey.pem ubuntu@<Server2_public_ip>
ping -c 4 10.0.1.10
```

13. Troubleshooting Common Issues

Symptom	Likely Cause	Fix
Ping/SSH times out across VPCs	Missing route table entry	Add route to peering connection on both sides
Ping fails, SSH works	Security group blocks ICMP	Add inbound ICMP rule from peer VPC CIDR
Peering stuck at 'Pending acceptance'	Not yet accepted by acceptor account	Accept from the acceptor VPC owner
Cannot create peering	Overlapping CIDR blocks	Use non-overlapping CIDR ranges
No internet access on instance	IGW not attached or no route to 0.0.0.0/0	Attach IGW and add default route

Note: The most common mistake is forgetting that a peering connection has **three** dependent pieces of configuration — the peering connection itself, the route table entries on both sides, and the security group rules. All three must be correct.

14. Cleanup — Deleting Resources

To avoid ongoing charges after completing this lab, delete the resources in the following order:

- Terminate **Server1_ADITYA** and **Server2_ADITYA** (EC2 → Instances → Terminate instance).
- Delete the peering connection **Peer_ADITYA** (VPC → Peering connections → Delete peering connection).
- Remove the routes referencing **Peer_ADITYA** from **RouteTable1_ADITYA** and **RouteTable2_ADITYA**.
- Detach and delete **IGW1_ADITYA** and **IGW2_ADITYA**.
- Delete **Subnet1_ADITYA** and **Subnet2_ADITYA**.
- Delete **VPC1_ADITYA** and **VPC2_ADITYA**.
- Delete the security groups **SG1_ADITYA** and **SG2_ADITYA** if they were not deleted with the VPCs.

15. Summary & Best Practices

In this guide we built two isolated VPCs, launched an Ubuntu 22.04 LTS EC2 instance in each, connected the VPCs using a VPC Peering connection, and verified private, cross-VPC connectivity between the two Ubuntu servers.

Key takeaways

- VPC Peering connects two VPCs privately without traversing the public internet.
- Peering is not transitive — each pair of VPCs that needs to talk requires its own connection.
- Route table entries are required on **both** sides of the peering connection.
- Security group rules must explicitly allow the traffic type (SSH, ICMP, etc.) from the peer VPC's CIDR range.
- CIDR blocks of peered VPCs must not overlap.
- For connecting many VPCs together, consider AWS **Transit Gateway** instead of a full mesh of peering connections.

*Document prepared by **ADITYA** — AWS VPC Peering Step-by-Step Guide*